SKILL 5,6 2520080009

```
krishnasree08@krishnasree08:~$ cd ~/ossp_ubuntu_topics
krishnasree08@krishnasree08:~/ossp_ubuntu_topics$ cat > skill5_topic1.c <<'EOF'
#include <stdio.h>
#include <string.h>

int main(void) {
    char input[200];
    printf("Enter text in single quotes: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    if (input[0] == '\'' && input[strlen(input)-1] == '\'') {
        input[strlen(input)-1] = '\0';
        printf("Literal content: %s\n", input + 1);
    } else {
        printf("Please enclose the text in single quotes.\n");
    }
    return 0;
}
EOF
krishnasree08@krishnasree08:~/ossp_ubuntu_topics$ gcc -Wall skill5_topic1.c -o skill5_1
./skill5_1
Enter text in single quotes: 'Hello $USER'
Literal content: Hello $USER
krishnasree08@krishnasree08:~/ossp_ubuntu_topics$ cat > skill5_topic2.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char input[200];
    printf("Enter text in double quotes: ");
    if (!fgets(input, sizeof(input), stdin)) return 1;
    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) < 2 ||
        input[0] != '"' || input[strlen(input)-1] != '"') {
        printf("Invalid quoted input.\n");
        return 1;
    }

    input[strlen(input)-1] = '\0';
    char *text = input + 1;
    char output[200] = "";
    char *p = text;

    while (*p) {
        if (*p == '$') {
            char name[100];
            int i = 0;
            p++;
            while ((*p >= 'A' && *p <= 'Z') ||
                   (*p >= 'a' && *p <= 'z') ||
                   (*p >= '0' && *p <= '9') || *p == '_') {
                if (i < 99) name[i++] = *p;
                p++;
            }
            name[i] = '\0';
            const char *value = getenv(name);
            if (value) strncat(output, value, sizeof(output)-strlen(output)-1);
        } else {
            int len = strlen(output);
            if (len < (int)sizeof(output)-1) {
                output[len] = *p;
```

```
    }

    printf("Parsed output: %s\n", output);
    return 0;
}
EOF
krishnasree08@krishnasree08:~/ossp_ubuntu_topics$ gcc -Wall skill5_topic2.c -o skill5_2
./skill5_2
Enter text in double quotes: "Hello $USER"
Parsed output: Hello krishnasree08
krishnasree08@krishnasree08:~/ossp_ubuntu_topics$ cat > skill5_topic3.c <<'EOF'
#include <stdio.h>
#include <string.h>

int main(void) {
    char input[200], output[200];
    int i = 0, j = 0;

    printf("Enter text with escape sequences: ");
    if (!fgets(input, sizeof(input), stdin)) return 1;

    input[strcspn(input, "\n")] = '\0';

    while (input[i] && j < 198) {
        if (input[i] == '\\' && input[i+1]) {
            i++;
            if (input[i] == 'n') output[j++] = '\n';
            else if (input[i] == 't') output[j++] = '\t';
            else output[j++] = input[i];
            i++;
        } else {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
    printf("Processed output:\n%s\n", output);
    return 0;
}
EOF
krishnasree08@krishnasree08:~/ossp_ubuntu_topics$ gcc -Wall skill5_topic3.c -o skill5_3
./skill5_3
Enter text with escape sequences: Hello\ World\nC\ Programming
Processed output:
Hello World
C Programming
krishnasree08@krishnasree08:~/ossp_ubuntu_topics$ cat > skill5_topic4.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process executing ls:\n");
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec failed");
```

```
    }

    if (pid == 0) {
        printf("Child process executing ls:\n");
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec failed");
        _exit(1);
    }

    int status;
    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid failed");
        return 1;
    }

    if (WIFEXITED(status))
        printf("Child exited with status %d\n", WEXITSTATUS(status));

    printf("Parent process completed.\n");
    return 0;
}
EOF
krishnasree08@krishnasree08:~/ossp_ubuntu_topics$ gcc -Wall skill5_topic4.c -o skill5_4
./skill5_4
Child process executing ls:
total 160
-rwxr-xr-x 1 krishnasree08 krishnasree08 16232 Oct 26 14:33 skill5_1
-rwxr-xr-x 1 krishnasree08 krishnasree08 16320 Oct 26 14:33 skill5_2
-rwxr-xr-x 1 krishnasree08 krishnasree08 16152 Oct 26 14:34 skill5_3
-rwxr-xr-x 1 krishnasree08 krishnasree08 16272 Oct 26 14:34 skill5_4
-rw-r--r-- 1 krishnasree08 krishnasree08   455 Oct 26 14:33 skill5_topic1.c
-rw-r--r-- 1 krishnasree08 krishnasree08  1285 Oct 26 14:33 skill5_topic2.c
-rw-r--r-- 1 krishnasree08 krishnasree08   684 Oct 26 14:34 skill5_topic3.c
-rw-r--r-- 1 krishnasree08 krishnasree08   644 Oct 26 14:34 skill5_topic4.c
-rwxr-xr-x 1 krishnasree08 krishnasree08 16280 Oct 26 14:26 topic1
-rw-r--r-- 1 krishnasree08 krishnasree08  1031 Oct 26 14:26 topic1_history.c
-rwxr-xr-x 1 krishnasree08 krishnasree08 16240 Oct 26 14:27 topic2
-rw-r--r-- 1 krishnasree08 krishnasree08  1879 Oct 26 14:27 topic2_memory.c
-rwxr-xr-x 1 krishnasree08 krishnasree08 16240 Oct 26 14:28 topic3
-rw-r--r-- 1 krishnasree08 krishnasree08   728 Oct 26 14:27 topic3_tokenizer.c
-rwxr-xr-x 1 krishnasree08 krishnasree08 16320 Oct 26 14:28 topic4
-rw-r--r-- 1 krishnasree08 krishnasree08  1574 Oct 26 14:28 topic4_parser.c
Child exited with status 0
Parent process completed.
krishnasree08@krishnasree08:~/ossp_ubuntu_topics$ gcc -Wall skill5_topic1.c -o skill5_1
gcc -Wall skill5_topic2.c -o skill5_2
gcc -Wall skill5_topic3.c -o skill5_3
gcc -Wall skill5_topic4.c -o skill5_4
krishnasree08@krishnasree08:~/ossp_ubuntu_topics$ ls
skill5_1  skill5_4         skill5_topic3.c  topic1_history.c  topic3              topic4_parser.c
skill5_2  skill5_topic1.c  skill5_topic4.c  topic2            topic3_tokenizer.c
skill5_3  skill5_topic2.c  topic1           topic2_memory.c   topic4
krishnasree08@krishnasree08:~/ossp_ubuntu_topics$
```

Result: Successfully implemented and executed four C programs in Ubuntu, demonstrating single and double quotes, escape sequences, and child process creation and execution.